

InfraThrone Interview-Readiness Docs (CoreOps Edition)

A complete pack that you will give to CoreOps learners so they can crack L1/L2 interviews confidently.

1. CoreOps Interview Themes (What companies expect)

CoreOps interviews test **foundational reliability knowledge**, not high-end architecture.

They expect you to:

Understand Linux basics

- processes
- CPU/memory I/O
- logs
- permissions
- services



Understand networking fundamentals

- DNS
- ports
- connectivity
- curl/nslookup
- service discovery

Understand Kubernetes “day-1 & day-2 operations”

- pods
- deployments
- services
- logs
- events
- basic troubleshooting

Understand SRE mindset

- incident mindset
- hypothesis-driven debugging
- first triage

Demonstrate basic production discipline

- structured debugging
- knowing what NOT to do
- careful with restarts/scales

CoreOps engineers must be **stable, consistent, reliable**, not flashy.

2. CoreOps Interview Question Bank (Week-1 Aligned)

(These are NOT solved, because learners must think.)

A. Linux & System Fundamentals

1. What is the difference between a process and a thread?
 2. How do you check CPU usage per process in Linux?
 3. What is dmesg used for in debugging?
 4. What causes an OOMKill?
 5. What is the difference between soft link and hard link?
 6. How do you check which process is listening on a port?
 7. What is the difference between stdout and stderr?
 8. How do you view the last 100 lines of a log file?
 9. What is the purpose of /etc/resolv.conf?
 10. What happens when disk is 100% full?
-

B. Networking & Troubleshooting

11. What is DNS and how does it work?
12. Difference between a port and a protocol?
13. What does curl -v actually show you?
14. How do you check if DNS resolution is working inside a pod?
15. How do you test connectivity between two pods?
16. What is the difference between ClusterIP and NodePort?

- 17. How does Kubernetes decide the IP of a service?
 - 18. Why would a service show "Endpoints: "?
 - 19. What is a readiness probe?
 - 20. What is a liveness probe?
-

C. Kubernetes Basics & Debugging

- 21. What happens in background when you do kubectl apply?
 - 22. Why do pods go into CrashLoopBackOff?
 - 23. Why do pods get stuck in Pending state?
 - 24. How do you check image pull issues?
 - 25. What is the difference between logs and events?
 - 26. How does Kubernetes decide which node a pod goes to?
 - 27. What is a ReplicaSet?
 - 28. What is the purpose of a Deployment?
 - 29. Why would a Pod be Running but not Ready?
 - 30. How do you view the environment variables inside a pod?
-

3. CoreOps Interview Checklists (What to Memorize)

These checklists train the learner to think like an operator rather than a developer.

A. Linux Debugging Checklist

- `top / htop`
- `df -h`
- `free -m`
- `dmesg | tail`
- `ps aux | grep`
- `netstat -tulnp`
- `journalctl -u <service>`

B. Kubernetes Debugging Checklist

- `kubectl logs`
 - `kubectl describe`
 - `kubectl get events`
 - `kubectl exec`
 - `kubectl get ep`
 - `kubectl top`
 - `kubectl get node -o wide`
-

C. Network Troubleshooting Checklist

- ping (if enabled)
 - curl
 - nc -zv
 - nslookup
 - Check service Endpoints
 - Check DNS resolution
 - Check NetworkPolicies
-

D. SRE-Mindset First-Triage Checklist

1. Is the issue widespread or isolated?
 2. Can it be reproduced?
 3. What changed recently?
 4. Check system health before app health
 5. Validate topology (pods → svc → endpoints → logs)
 6. Document everything
-

4. CoreOps Interview “Answer Framing Guide.”

Interviewers judge based on **how you explain**, not just what you know.

Teach CoreOps learners to answer using:

The InfraThrone 3-Step Answer Pattern

- ✓ Step 1: Define the concept in simple English
 - ✓ Step 2: Connect it to production behavior
 - ✓ Step 3: Provide 1–2 real debugging commands
-

Example: *“What is readiness probe?”*

Step 1: Readiness probe decides if the app is ready to serve traffic.

Step 2: If it fails, Kubernetes removes the pod from service endpoints.

Step 3:

1. `kubectl describe pod <pod>`
2. `kubectl logs <pod>`

This pattern makes even beginners sound like L2 engineers.
